

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name:	Saparali Adina	Student ID:	230107064
Hardware / CPU:	AMD Ryzen 7 5800HS with Radeon Graphics	Date & Section:	02.10.2026



LAB PROTOCOL & AUTONOMOUS RULES OF ENGAGEMENT

- **Autonomous Session:** You are expected to read, run, optimize, and document all findings independently.
- **Zero Support for Dependency Issues:** If you encounter an error, verify that you installed packages inside a clean virtual environment. Compilers are NOT needed for this lab.
- **Equal Hardware Standard:** Keep your laptop plugged into wall power and close background browsers, video streams, or games to prevent CPU thermal throttling and frequency capping.
- **Whiteboard Leaderboard:** When you finish Task 1 and Task 3, record your CPU model and best speedup factor on the lab whiteboard.

1. Background & Environment Setup (10 Minutes)

Standard CPython executes bytecode under the Global Interpreter Lock (GIL), preventing multiple CPU threads from executing simultaneously in Python space. In this lab, we bypass the GIL using Numba's JIT compiler with its native multithreaded OpenMP/TBB work-stealing engine. Because Numba compiles numeric Python loops directly to machine code and releases the GIL inside parallel loops (prange), we achieve bare-metal native CPU scaling across both macOS (Apple Silicon M-series) and Windows/Linux x86_64 machines with zero local C++ compiler dependencies.

Execute the following in your terminal / command prompt:

```
pip install numpy numba matplotlib
```

2. Challenge 1: The Core Speedometer & Amdahl's Law

Objective: Benchmark an embarrassingly parallel Monte Carlo simulation to approximate π across varying thread counts. You will measure execution latency, quantify parallel speedup, and observe where Amdahl's Law caps performance.

```
import time
import numpy as np
import numba
from numba import njit, prange
print(f"Hardware Threads Detected: {numba.config.NUMBA_NUM_THREADS}")
@njit(parallel=True)
def
    monte_carlo_pi(n_samples)
    : inside_circle = 0
for i in prange(n_samples):
x = np.random.uniform(0.0, 1.0) y =
    np.random.uniform(0.0, 1.0) if
    x * x + y * y <= 1.0:
inside_circle += 1
return (4.0 * inside_circle) / n_samples

# JIT Warmup (excludes JIT compilation overhead from benchmarks)
_ = monte_carlo_pi(10_000)

SAMPLES = 120_000_000
thread_counts = [1, 2, 4, 8, numba.config.NUMBA_NUM_THREADS]
# Filter out duplicates and invalid counts
thread_counts = sorted(list(set([t for t in thread_counts if t <=
    numba.config.NUMBA_NUM_THREADS])))

print(f"{'Threads':<10} | {'Time (s)':<12} | {'Speedup':<10} | {'Efficiency
    (%)':<15}")
print("-" * 55)

t1_baseline = None
for t in thread_counts:
    numba.set_num_threads(t)
    start =
    time.perf_counter()
pi_est = monte_carlo_pi(SAMPLES) elapsed =
    time.perf_counter() - start

if t == 1:
t1_baseline = elapsed speedup =
    1.0
    efficiency = 100.0
else:
```

```

speedup = t1_baseline / elapsed efficiency
          = (speedup / t) * 100.0

print(f"{t:<10} | {elapsed:<12.4f} | {speedup:<10.2f}x |
{efficiency:<15.1f}")

```

Deliverables & Technical Questions for Challenge 1:

- A. Record your baseline T₁ (1 thread) and your maximum multi-thread runtime T_{max} in Table 1 below.

Threads	Time (s)	Speedup	Efficiency
1	4.8019	1.00×	100.0%
2	2.6478	1.81×	90.7%
4	1.7373	2.76×	69.1%
8	0.9160	5.24×	65.5%
16	0.6321	7.60×	47.5%

T₁ = 4.8019 s

T_{max} (16 threads) = 0.6321 s

Maximum speedup = 7.60x

- B. Why is the warm-up call `_ = monte_carlo_pi(10_000)` strictly required before recording benchmark timings?

The warm-up is needed because Numba compiles the function when it runs for the first time. If we measure the first run, the compilation time will also be included. So, we do the warm-up first to get more accurate benchmark results.

- C. Did your parallel efficiency stay at 100% when scaling from 1 to max threads? What architectural factors (e.g. memory bus contention, hyperthreading/SMT core vs physical core limits) explain the degradation?

No, the efficiency decreased when I increased the number of threads. It was 100% with 1

thread, but with 16 threads it dropped to 47.5%. It happens because using more threads also creates some overhead, and CPU and memory resources are limited. Also, logical threads do not give the same performance as physical CPU cores.

- D. Why does `inside_circle += 1` inside `prange` not corrupt the count? What implicit OpenMP reduction construct does Numba generate here?

It does not corrupt the count because Numba recognizes it as a reduction operation inside `prange`. Each thread calculates its own part of the result, and Numba combines these results safely at the end.

3. Challenge 2: Load Imbalance & Dynamic Scheduling

Objective: Compute a high-resolution 2500×2500 Mandelbrot fractal. Unlike Monte Carlo, points in the complex plane require radically different amounts of compute: points inside the set iterate all the way to `max_iter`, while outer points escape in 2-3 iterations. You will investigate how non-uniform workloads impact thread utilization.

```
import time
import numpy as np
import matplotlib.pyplot as plt
from numba import njit, prange

@njit(parallel=True)
def render_mandelbrot_rows(h, w,
                           max_iter):
    img = np.zeros((h, w),
                   dtype=np.int32) # Row-parallel loop
    for r in prange(h):
        cy = -1.2 + (r / h) * 2.4
        for c in prange(w):
            cx = -2.0 + (c / w) * 2.5
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                next_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                z_imag = next_imag
                it += 1
            img[r, c] = it
    return img

@njit(parallel=True)
def render_mandelbrot_cols(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Column-parallel loop (swapped iteration axis) for
    for c in prange(w):
        cx = -2.0 + (c / w) * 2.5
        for r in prange(h):
            cy = -1.2 + (r / h) * 2.4
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                next_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                z_imag = next_imag
                it += 1
```

```

        img[r, c] = it
    return img

# Warmup JIT
_ = render_mandelbrot_rows(100, 100, 50)
_ = render_mandelbrot_cols(100, 100, 50)

H, W, MAX_IT = 2500, 2500, 1000

t0 = time.perf_counter()
grid_rows = render_mandelbrot_rows(H, W,
MAX_IT) t_rows = time.perf_counter() - t0

t1 = time.perf_counter()
grid_cols = render_mandelbrot_cols(H, W,
MAX_IT) t_cols = time.perf_counter() - t1

print(f"Row-Parallel Render          {t_rows:.3f}
Time: s")
print(f"Column-Parallel Render Time: {t_cols:.3f} s")

# Save visual artifact
plt.figure(figsize=(8, 8))
plt.imshow(grid_rows, cmap='magma', extent=[-2.0, 0.5, -1.2,
1.2]) plt.title(f"Mandelbrot {H}x{W} (Render: {t_rows:.2f}s)")
plt.axis('off')
plt.savefig('mandelbrot_output.png', dpi=300,
bbox_inches='tight')
print("Saved image: mandelbrot_output.png")

```

Deliverables & Technical Questions for Challenge 2:

- A. Compare `t_rows` vs `t_cols` . Which axis decomposition completed faster on your machine and why?

On my laptop, the row-parallel version was a little faster. The row version took 1.492 seconds, while the column version took 1.547 seconds. s. I think the row version was faster because NumPy stores arrays row by row, so accessing the data this way is more efficient.

- B. Explain how NumPy's C-contiguous (row-major) memory layout interacts with CPU cache lines when threads write to `img[r, c]` vs `img[c, r]` .

NumPy normally stores arrays in row-major order, which means values from the same row are next to each other in memory. Because of this, reading and writing row by row works better with the CPU cache. Column access has to jump between different memory locations, so it can be a little slower.

- C. Load imbalance analysis: If thread 0 receives the top 20% of rows and thread 2 receives the

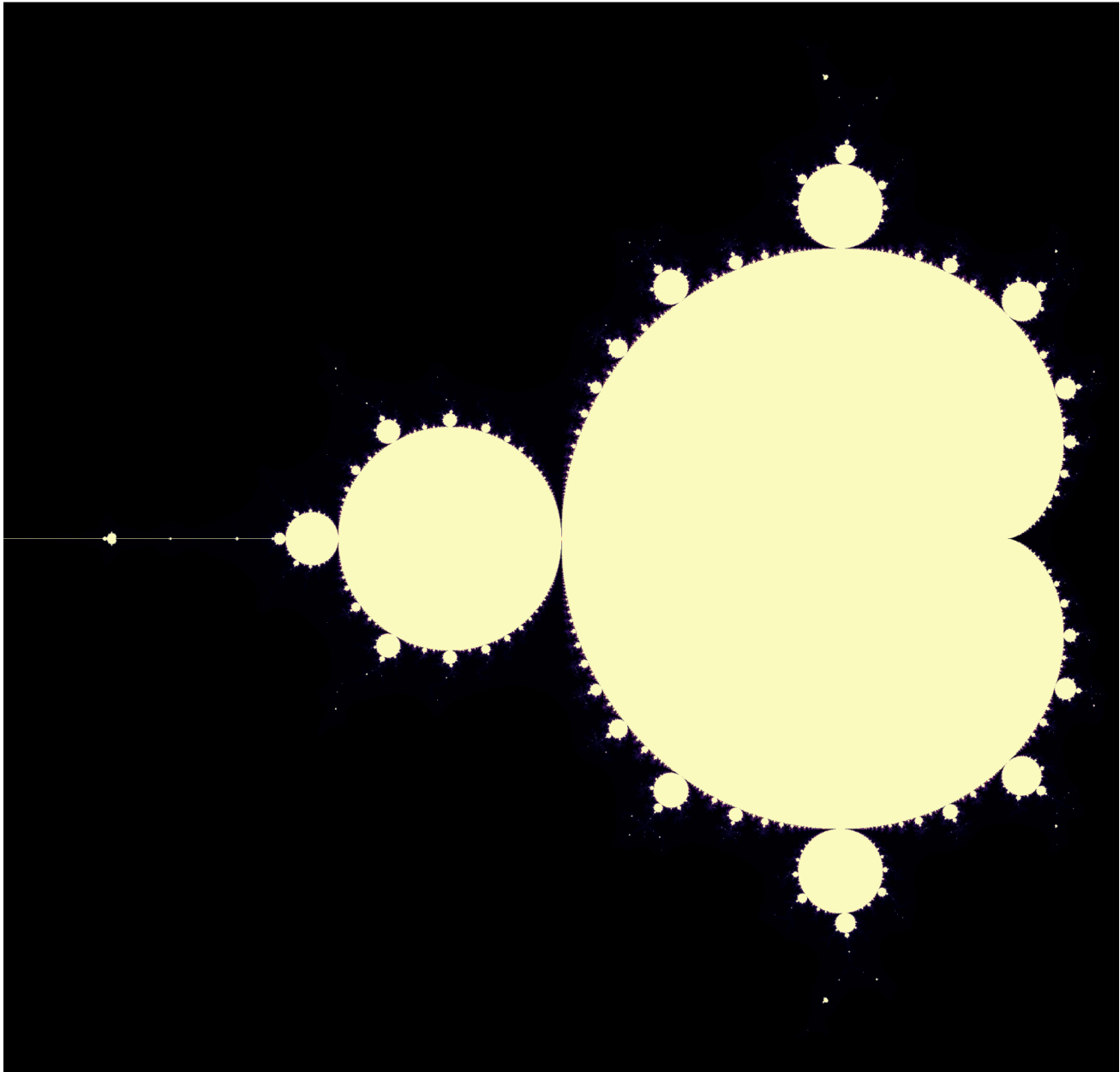
Instructor – sufyan.mustafa@sdu.edu.kz

center 20% of rows, why does one thread idle while the other struggles? How does OpenMP dynamic scheduling resolve this?

The workload is not equal because some Mandelbrot points need only a few iterations, while other points need many more iterations. Because of this, one thread can finish its rows early while another thread is still working. Dynamic scheduling can help because new work can be given to threads when they become free.

- D. Attach your exported mandelbrot_output.png file alongside your completed task sheet.

Mandelbrot 2500x2500 (Render: 1.49s)



4. Challenge 3: Stencil Computation & Memory Bandwidth

Objective: Simulate 2D heat diffusion using a 5-point discrete Laplacian stencil over 300 timesteps on a 1500×1500 grid. Unlike the compute-bound fractal, stencil computations are strictly memory-bandwidth bound. You will determine whether adding CPU cores continues to speed up performance or hits the RAM memory bus wall.

```
import time
import numpy as np
from numba import njit, prange

@njit(parallel=True)
def heat_step(u, u_next, alpha=0.20):
    rows, cols = u.shape
    for i in prange(1, rows - 1):
        for j in range(1, cols - 1):
            u_next[i, j] = u[i, j] + alpha * (
                u[i+1, j] + u[i-1, j] + u[i, j+1] + u[i, j-1] - 4.0 * u[i, j]
            )

GRID_SIZE = 1500
STEPS = 300

u = np.zeros((GRID_SIZE, GRID_SIZE), dtype=np.float64)
u_next = np.zeros_like(u)

# Dirichlet Boundary conditions: Top & Left walls maintained at 100°C
u[0, :] = 100.0
u[:, 0] = 100.0
u_next[0, :] = 100.0
u_next[:, 0] = 100.0

# Warmup
heat_step(u, u_next)

# Benchmark Execution
start = time.perf_counter()
for step in range(STEPS):
    heat_step(u, u_next)
# Pointer swap (avoids array allocation copies) u,
    u_next = u_next, u
elapsed = time.perf_counter() - start

cells_per_sec = (GRID_SIZE * GRID_SIZE * STEPS) / elapsed / 1e6
print(f"Heat Diffusion Complete: {elapsed:.3f} s")
print(f"Throughput: {cells_per_sec:.2f} Megacells/sec")
```


Deliverables & Technical Questions for Challenge 3:

- A. Record your benchmark runtime and Megacells/sec throughput in Table 1.

My benchmark runtime was 0.687 seconds, and the throughput was 982.71 Megacells/sec.

- B. Change the array precision from np.float64 to np.float32 . Re-run the test. By what factor did runtime decrease? Why does halving memory footprint improve stencil throughput far more than compute capability?

With float64, the runtime was 0.687 seconds. After changing it to float32, the runtime decreased to 0.216 seconds. So, float32 was about 3.18 times faster in my test. The throughput also increased from 982.71 to 3120.33 Megacells/sec. This happens because float32 uses less memory than float64, so less data has to be moved between RAM and the CPU.

- C. Memory Roofline Model: Explain why adding twice as many CPU cores does not produce a 2x speedup on stencil codes once memory bandwidth is saturated.

Adding twice as many CPU cores does not always make the program two times faster. This calculation depends a lot on memory bandwidth. When the memory bandwidth is already fully used, adding more CPU cores does not help much because all the cores still have to share the same memory bandwidth.

5. Student Result Scorecard & Data Collection

Fill in your benchmark results into Table 1 below before submitting this document:

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	4.8019	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	2.6478	1.81x	90.7%
Challenge 1: Monte Carlo	4 Threads	120,000,000	1.7373	2.76x	69.1%
Challenge 1: Monte Carlo	Max Threads	120,000,000	0.6321	7.60x	47.5%
Challenge 2: Mandelbrot (Rows)	Max Threads	2500 x 2500	1.492	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads	2500 x 2500	1.547	N/A	N/A
Challenge 3: Heat Stencil	Max Threads	1500 x 1500 x 300	0.687	N/A	N/A